Visual Programming

C# (C sharp)

C# Applications
Variables & Data Types

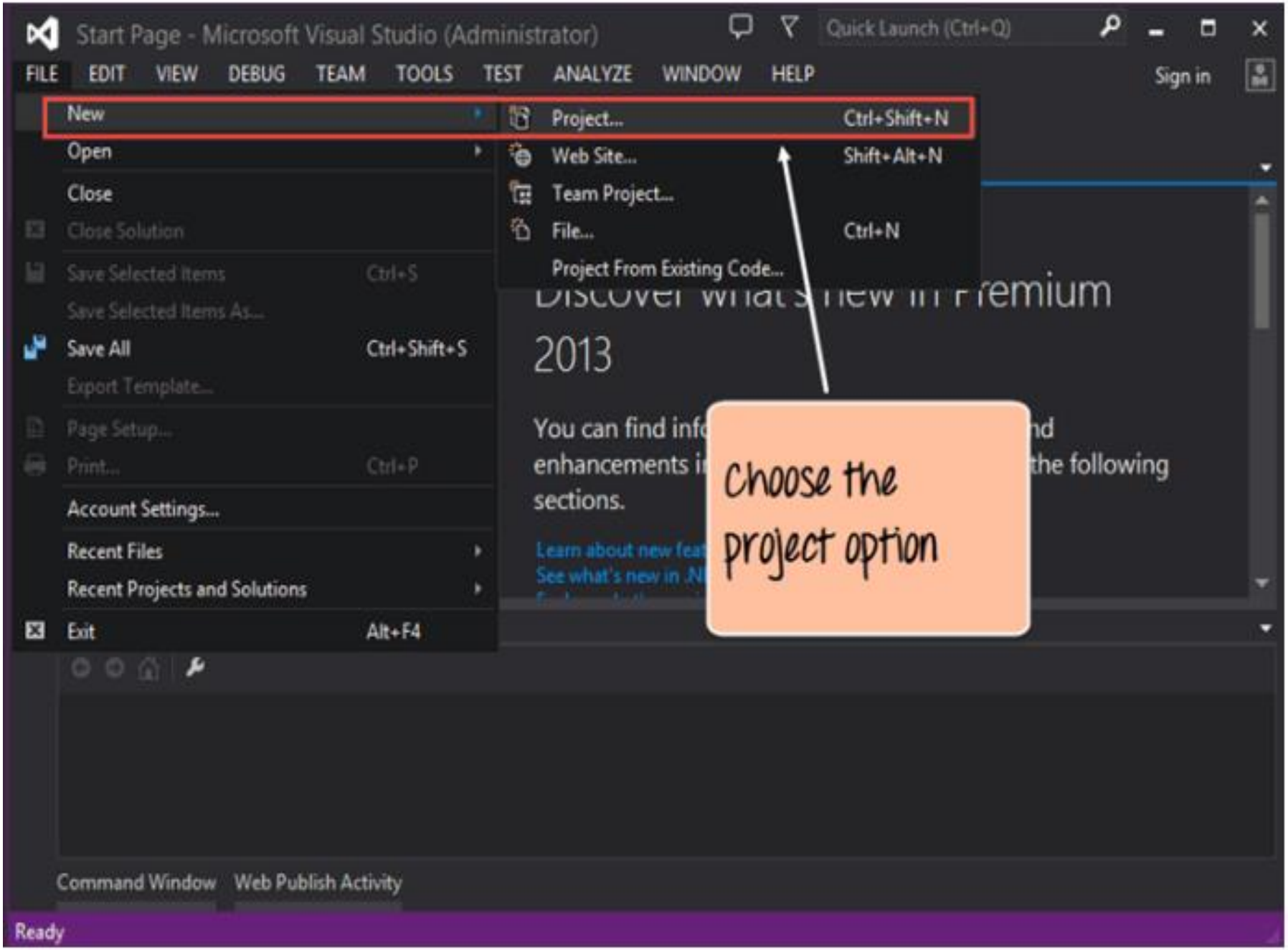
Lecturer: Dr. Khaled Mohammed BinAbdl

Department of Information Technology,

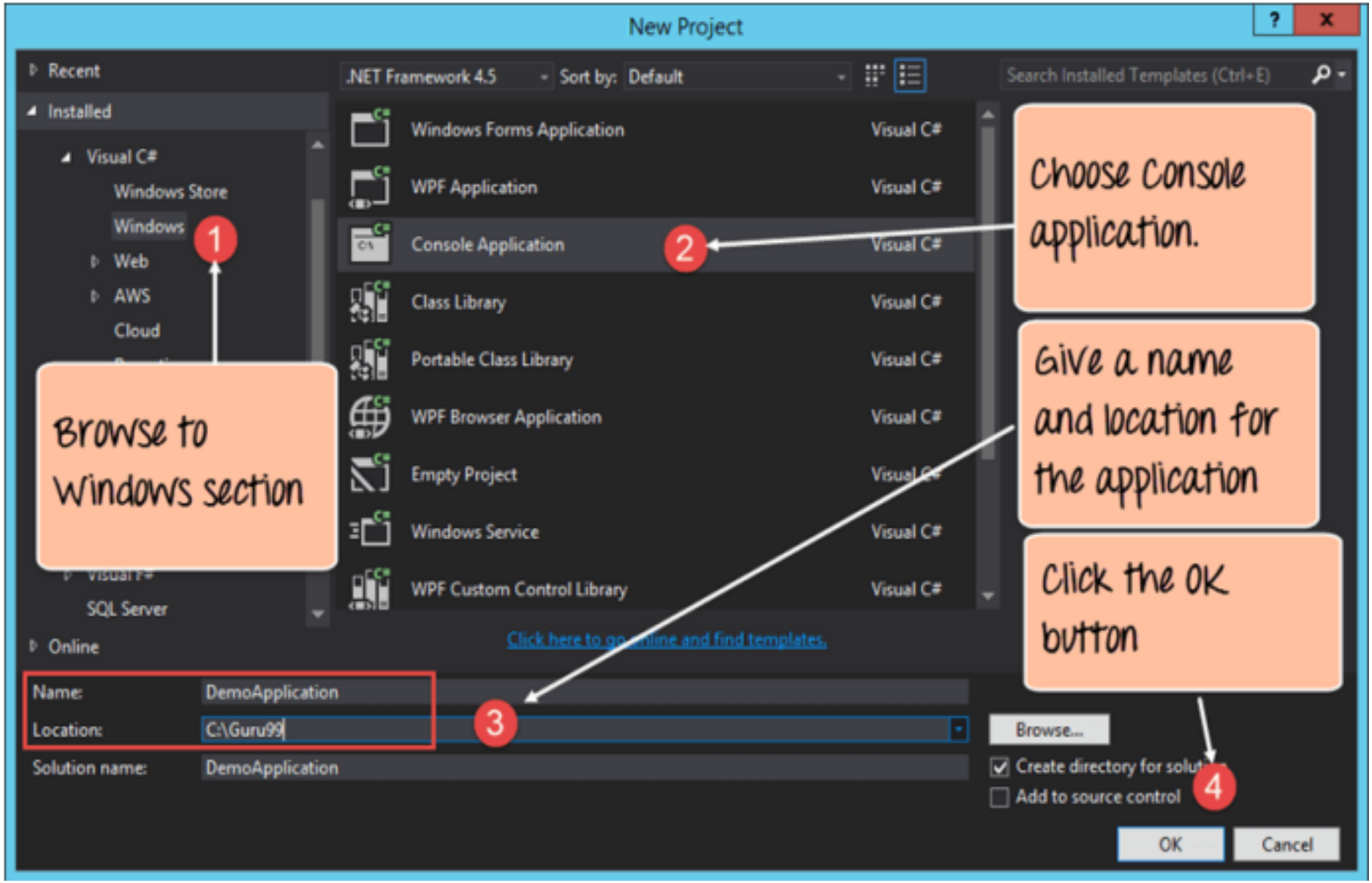
College of Computers, Seiyun University

Building the first console application

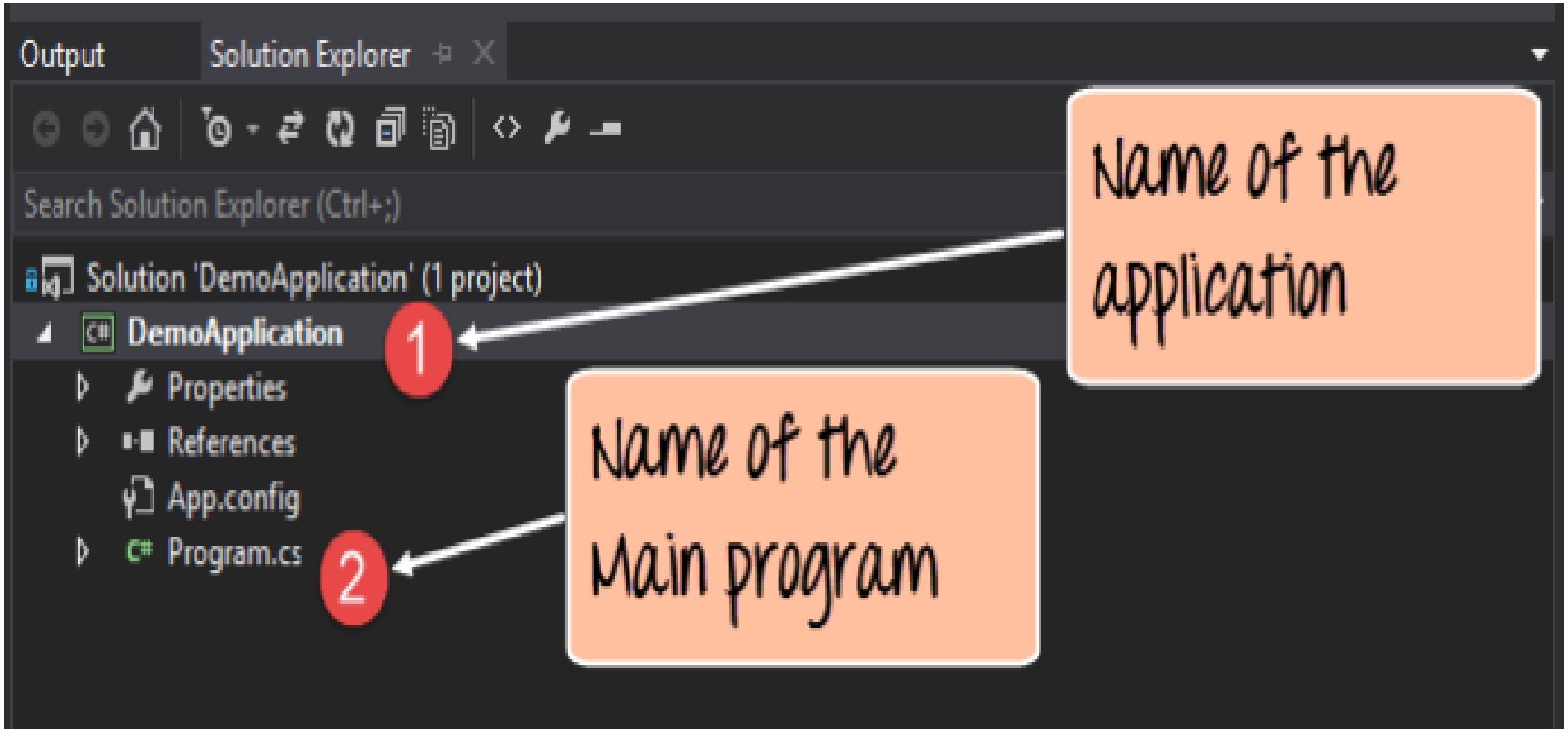
- A console application is an application that can be run in the command prompt in Windows.
- Steps:
  - Step 1) The first step involves the creation of a new project in Visual Studio. For that, once the Visual Studio is launched, you need to choose the menu option New->Project.


Start Page - Microsoft Visual Studio (Administrator)
Quick Launch (Ctrl+Q)
FILE
EDIT
VIEW
DEBUG
TEAM
TOOLS
TEST
ANALYZE
WINDOW
HELP
Sign in
New
Open
Close
Close Solution
Save Selected Items Ctrl+S
Save Selected Items As...
Save All Ctrl+Shift+S
Export Template...
Page Setup...
Print... Ctrl+P
Account Settings...
Recent Files
Recent Projects and Solutions
Exit Alt+F4
Project... Ctrl+Shift+N
Web Site... Shift+Alt+N
Team Project...
File... Ctrl+N
Project From Existing Code...
2013
You can find info
enhancements i
sections.
the following
Choose the project option
Command Window
Web Publish Activity
Ready

Step 2) choose the project type as a Console application. Follow instructions drawn on figure

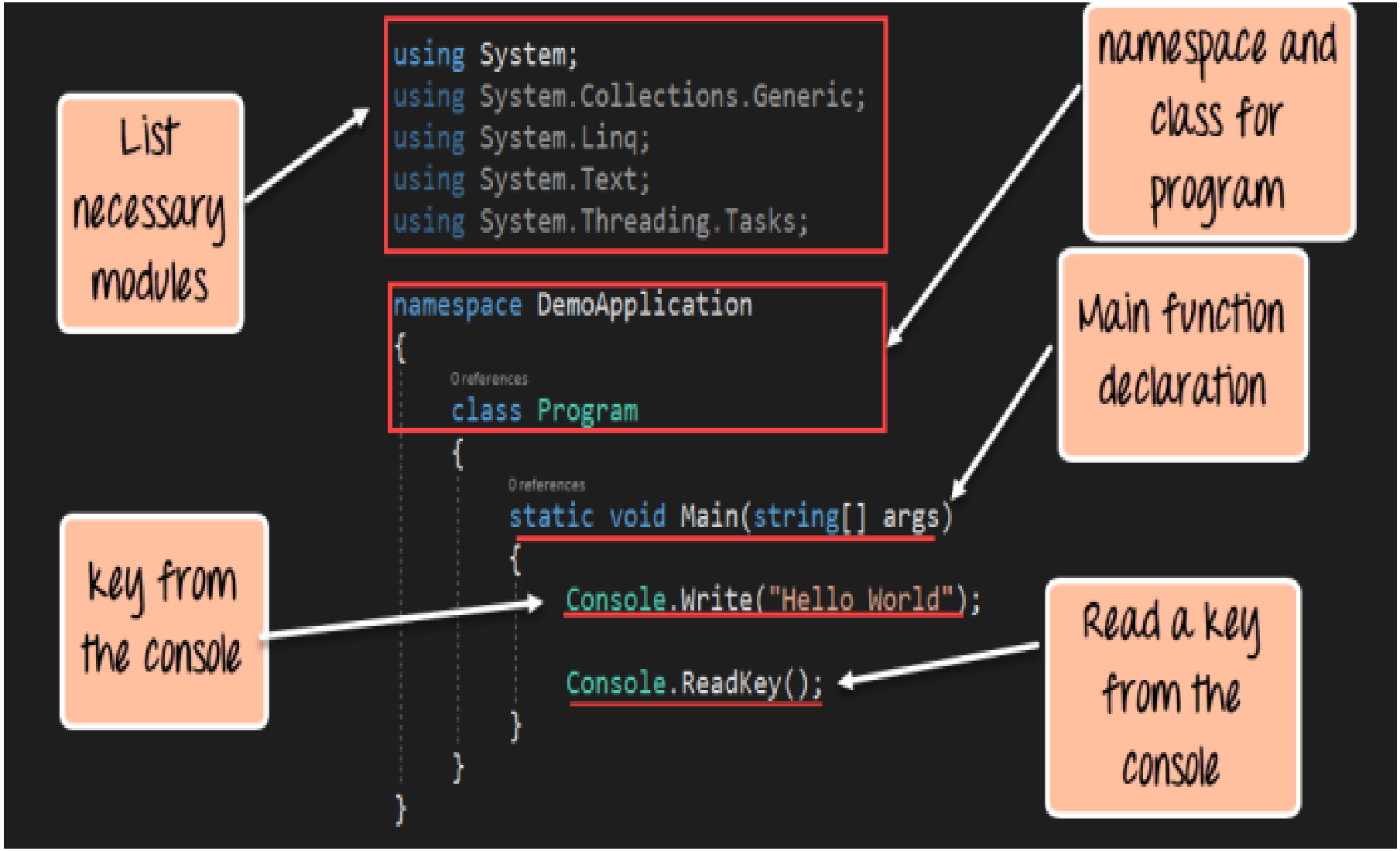


The below output in Visual Studio

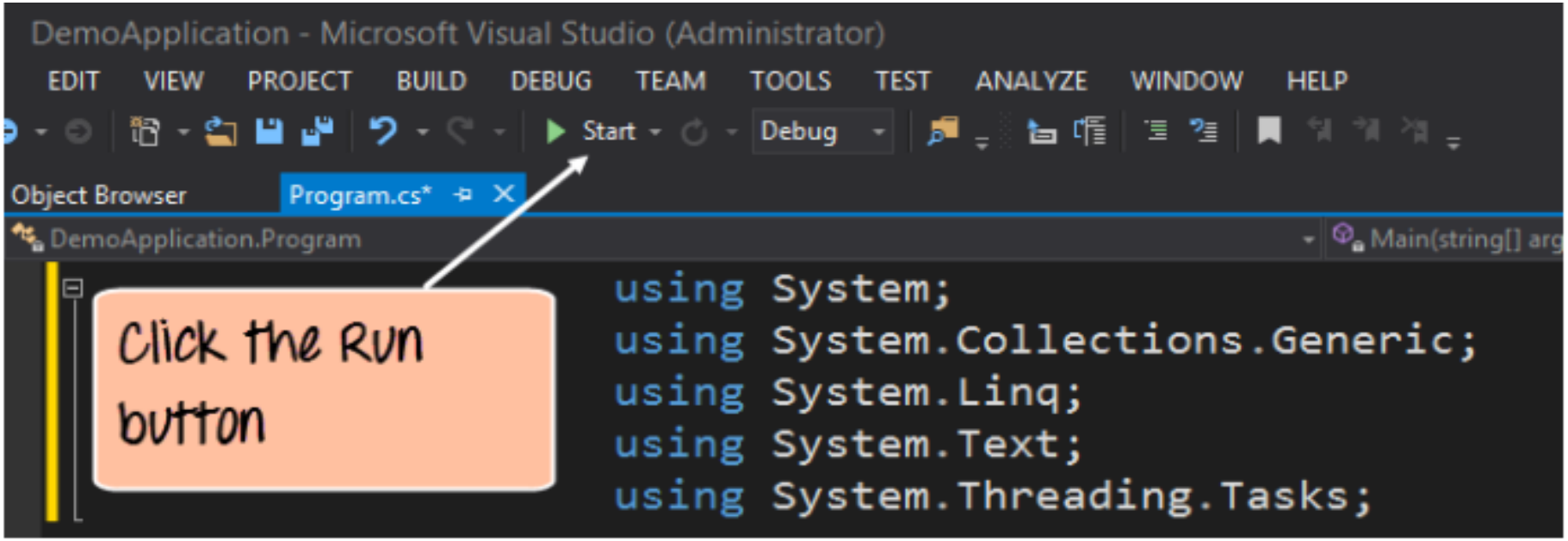


1. A project called 'DemoApplication' will be created in Visual Studio. This project will contain all the necessary artifacts required to run the Console application.
2. The Main program called Program.cs is default code file which is created when a new application is created in Visual Studio. This code will contain the necessary code for the console application.

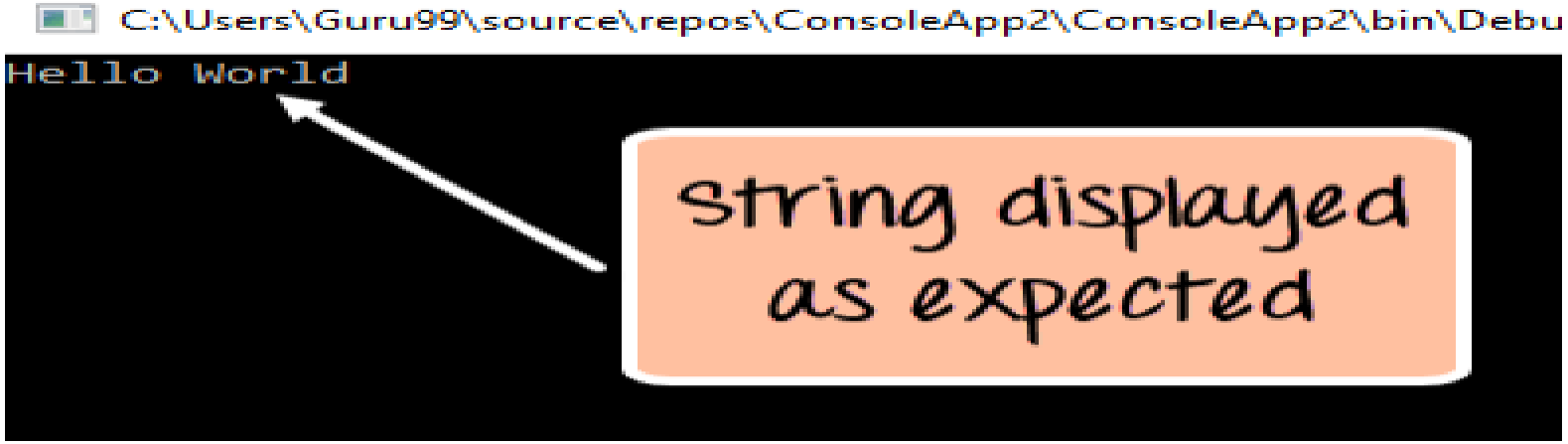
Step 3) Now let's write our code which will be used to display the string "Hello World" in the console application.



Step 4) Run your .Net program. To run any program, you need to click the Start button in Visual Studio.



Output:



Variables & Data Types - Variables

- Variables are concerned with the storage of data. Essentially.
- The C# syntax for declaring variables simply involves specifying the type and variable name as follows:
  - <type> <name>;
    - int age;
- Variable Naming
  - The basic variable naming rules are as follows:
  - The first character of a variable name must be either a letter, an underscore character ("_"), or "@"
  - Subsequent characters may be letters, underscore characters, or numbers

Keywords and contextual keywords					
abstract	as	base	bool	break	byte
case	catch	char	checked	class	const
continue	decimal	default	delegate	do	double
else	enum	event	explicit	extern	false
finally	fixed	float	for	foreach	goto
if	implicit	in	int	interface	internal
is	lock	long	namespace	new	null
object	operator	out	override	params	private
protected	public	readonly	ref	return	sbyte
sealed	short	sizeof	stackalloc	static	string
struct	switch	this	throw	true	try
typeof	uint	ulong	unchecked	unsafe	ushort
using	virtual	void	volatile	while	

Fig. 3.2 | Keywords and contextual keywords. (Part 1 of 2.)

Keywords and contextual keywords					
Contextual Keywords					
`add`	`alias`	`ascending`	`async`	`await`	`by`
`descending`	`dynamic`	`equals`	`from`	`get`	`global`
`group`	`into`	`join`	`let`	`nameof`	`on`
`orderby`	`partial`	`remove`	`select`	`set`	`value`
`var`	`where`	`yield`			

Fig. 3.2 | Keywords and contextual keywords. (Part 2 of 2.)

Variables & Data Types -
Data Types

- There are 13 simple Data types in C#.

  `char`, `byte`, `sbyte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `float`, `double`, decimal and bool.

- Instance variables of types are all given the value 0 by default except Instance variables of bool are given the value false by default.
- C# requires all variables to have a type.
- Reference-type instance variables are initialized by default to the value `null`.

Data Types…

Type	Alias for	Allowed Values
sbyte	System.SByte	Integer between –128 and 127.
byte	System.Byte	Integer between 0 and 255.
short	System.Int16	Integer between –32768 and 32767.
ushort	System.UInt16	Integer between 0 and 65535.
int	System.Int32	Integer between –2147483648 and 2147483647.
uint	System.UInt32	Integer between 0 and 4294967295.
long	System.Int64	Integer between – 9223372036854775808 and 9223372036854775807.
ulong	System.UInt64	Integer between 0 and 18446744073709551615.

Data Types…

Type	Alias for	Approx Min Value	Approx Max Value
`float`	`System.Single`	1.5×10^{-45}	3.4×10^{38}
`double`	`System.Double`	5.0×10^{-324}	1.7×10^{308}
`decimal`	`System.Decimal`	1.0×10^{-28}	7.9×10^{28}

Type	Alias for	Allowed Values
`char`	`System.Char`	Single Unicode character, stored as an integer between 0 and 65535.
`bool`	`System.Boolean`	Boolean value, `true` or `false`.
`string`	`System.String`	A sequence of characters.

Escape sequence	Description
`\n`	Newline. Positions the screen cursor at the beginning of the next line.
`\t`	Horizontal tab. Moves the screen cursor to the next tab stop.
`\"`	Double quote. Used to place a double-quote character (") in a string—e,g., `Console.Write("\"in quotes\"");` displays `"in quotes"`.
`\r`	Carriage return. Positions the screen cursor at the beginning of the current line—does not advance the cursor to the next line. Any characters output after the carriage return overwrite the characters previously output on that line.
`\\`	Backslash. Used to place a backslash character in a string.

Fig. 3.12 | Common escape sequences.

Integer, Double and Boolean Example

```
using System;
using System.Collections.Generic;
using System.Text;
using System.Threading.Tasks;
namespace DemoApplication
 {
     class Program
    {
     static void Main(string[] args)
       {
           Int num1 = 30;
           double num2 = 30.33;
           status = true;
           Console.Write(num1);
           Console.Write(num2);
           Console.Write(status);
           Console.ReadKey();
       }
    }
  }
```

String

A String data type is used to work with String values. In C#, the datatype is denoted by the keyword ‘String’.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace DemoApplication
  {
     class Program
       {
           static void Main(string[] args)
             {
                String message="Hello";
                Console.Write(message);
                Console.ReadKey();
             }
         }
  }
```

C# Enumeration

- An enumeration is used in any programming language to define a constant set of values.
- For example, the days of the week can be defined as an enumeration and used anywhere in the program.
- In C#, the enumeration is defined with the help of the keyword ‘enum’.
- define an enumeration called days, which will be used to store the days of the week.

C# Enumeration…

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace DemoApplication
{
   class Program
    {
        enum Days{Sun,Mon,tue,Wed,thu,Fri,Sat};
        static void Main(string[] args)
            {
                Console.Write(Days.Sun);
                Console.ReadKey();
             }
    }
 }
```